



Bikkad IT Institute

Session 5.2: Python Sets

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Python Programming - Beginner Level Notes

Prepared for: Bikkad IT Institute Students

Topic: Python Sets

Date: February 2026

1. Create a Set

What is a Set?

A set is an unordered collection of unique items. Sets automatically remove duplicates.

Key Features:

- Unordered - No index positions
- Unique items - Duplicates automatically removed
- Mutable - Can add/remove items
- No duplicate values
- Defined using curly braces { }

Code Examples

Example 1: Different Ways to Create Set

Empty set (must use set(), not {})

```
empty_set = set()
print(type(empty_set)) # <class 'set'>
```

Set with values

```
numbers = {1, 2, 3, 4, 5}
print(numbers) # {1, 2, 3, 4, 5}
```

Duplicates automatically removed

```
my_set = {1, 2, 2, 3, 3, 3, 4}
print(my_set) # {1, 2, 3, 4}
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Mixed data types

```
mixed = {1, "Python", 3.14, True}
print(mixed)
```

Example 2: Create Set from Other Collections

From list

```
my_set = set([1, 2, 3, 4, 5])
print(my_set) # {1, 2, 3, 4, 5}
```

From tuple

```
my_set = set((1, 2, 3, 4))
print(my_set)
```

From string (creates set of characters)

```
char_set = set("hello")
print(char_set) # {'h', 'e', 'l', 'o'}
```

Remove duplicates from list

```
numbers = [1, 2, 2, 3, 3, 3, 4, 4]
unique_numbers = set(numbers)
print(unique_numbers) # {1, 2, 3, 4}
```

2. Accessing Items

Important Note

Sets are unordered - cannot access items by index.

Code Examples

Example 1: Cannot Access by Index

```
my_set = {1, 2, 3, 4, 5}
```

This will cause ERROR

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

`print(my_set[0])` # TypeError: 'set' object is not subscriptable

Example 2: Check Membership

```
fruits = {"Apple", "Banana", "Mango", "Orange"}
```

```
print("Apple" in fruits) # True
print("Grapes" in fruits) # False
print("Grapes" not in fruits) # True
```

Example 3: Loop Through Set

```
fruits = {"Apple", "Banana", "Mango"}
```

Using for loop

```
for fruit in fruits:
    print(fruit)
```

Using while loop with conversion to list

```
fruit_list = list(fruits)
i = 0
while i < len(fruit_list):
    print(fruit_list[i])
    i += 1
```

Example 4: Check if Set is Empty

```
my_set = set()
```

```
if my_set:
    print("Set has items")
else:
    print("Set is empty")
```

```
if len(my_set) == 0:
    print("Set is empty")
```

3. Editing Items

Important Note

Cannot change individual items because sets are unordered. Can only add or remove entire items.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Code Examples

Example 1: Cannot Edit Items Directly

```
my_set = {1, 2, 3, 4, 5}
```

This will cause ERROR

```
my_set[0] = 100 # TypeError: 'set' object  
does not support item assignment
```

Example 2: Add Items

```
fruits = {"Apple", "Banana"}
```

Add single item

```
fruits.add("Mango")  
print(fruits) # {'Apple', 'Banana', 'Mango'}
```

Example 3: Update (Add Multiple Items)

```
fruits = {"Apple", "Banana"}
```

Add multiple items from list

```
fruits.update(["Mango", "Orange"])  
print(fruits) # {'Apple', 'Banana', 'Mango', 'Orange'}
```

Add multiple items from another set

```
fruits.update({"Grapes", "Kiwi"})  
print(fruits)
```

Add from tuple

```
fruits.update(("Papaya", "Guava"))  
print(fruits)
```

Example 4: Replace by Removing and Adding

```
numbers = {1, 2, 3, 4, 5}
```

OUR LOCATIONS

Remove old value

```
numbers.remove(3)
```

Add new value

```
numbers.add(30)  
print(numbers) # {1, 2, 30, 4, 5}
```

4. Delete Items

Method	Description
remove(item)	Remove item (error if not found)
discard(item)	Remove item (no error if not found)
pop()	Remove and return random item
clear()	Remove all items
del	Delete entire set

Table 1: Methods to Delete Items

Code Examples

Example 1: remove() Method

```
fruits = {"Apple", "Banana", "Mango", "Orange"}
```

Remove specific item

```
fruits.remove("Banana")  
print(fruits) # {'Apple', 'Mango', 'Orange'}
```

Remove item that doesn't exist (ERROR)

```
fruits.remove("Grapes") # KeyError:  
'Grapes'
```

Example 2: discard() Method

```
fruits = {"Apple", "Banana", "Mango"}
```

OUR LOCATIONS

Remove specific item

```
fruits.discard("Banana")  
print(fruits) # {'Apple', 'Mango'}
```

Remove item that doesn't exist (NO ERROR)

```
fruits.discard("Grapes") # No error  
print(fruits)
```

Example 3: pop() Method

```
numbers = {1, 2, 3, 4, 5}
```

Remove and return random item

```
removed = numbers.pop()  
print("Removed:", removed)  
print("Set now:", numbers)
```

Pop from empty set causes error

```
empty_set = set()
```

```
empty_set.pop() # KeyError: 'pop from an  
empty set'
```

Example 4: clear() Method

```
fruits = {"Apple", "Banana", "Mango"}
```

Remove all items

```
fruits.clear()  
print(fruits) # set()
```

Example 5: del Keyword

```
my_set = {1, 2, 3, 4, 5}
```

Delete entire set

```
del my_set
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

`print(my_set)` # **NameError: name 'my_set' is not defined**

5. Set Operations

Mathematical Set Operations

Operation	Operator	Method
Union		union()
Intersection	&	intersection()
Difference	-	difference()
Symmetric Diff	$\wedge$	symmetric_difference()

Table 2: Set Operations

Code Examples

Example 1: Union (All Items)

```
set1 = {1, 2, 3, 4}
```

```
set2 = {3, 4, 5, 6}
```

Using | operator

```
result = set1 | set2
```

```
print(result) # {1, 2, 3, 4, 5, 6}
```

Using union() method

```
result = set1.union(set2)
```

```
print(result) # {1, 2, 3, 4, 5, 6}
```

Union of multiple sets

```
set3 = {7, 8, 9}
```

```
result = set1 | set2 | set3
```

```
print(result) # {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

Example 2: Intersection (Common Items)

```
set1 = {1, 2, 3, 4}
```

```
set2 = {3, 4, 5, 6}
```

OUR LOCATIONS

Using & operator

```
result = set1 & set2  
print(result) # {3, 4}
```

Using intersection() method

```
result = set1.intersection(set2)  
print(result) # {3, 4}
```

Example 3: Difference (Items in First Set Only)

```
set1 = {1, 2, 3, 4}  
set2 = {3, 4, 5, 6}
```

Items in set1 but not in set2

```
result = set1 - set2  
print(result) # {1, 2}
```

Using difference() method

```
result = set1.difference(set2)  
print(result) # {1, 2}
```

Items in set2 but not in set1

```
result = set2 - set1  
print(result) # {5, 6}
```

Example 4: Symmetric Difference (Not in Both)

```
set1 = {1, 2, 3, 4}  
set2 = {3, 4, 5, 6}
```

Items in either set but not in both

```
result = set1 ^ set2  
print(result) # {1, 2, 5, 6}
```

Using symmetric_difference() method

```
result = set1.symmetric_difference(set2)  
print(result) # {1, 2, 5, 6}
```

Example 5: Subset and Superset

```
set1 = {1, 2, 3, 4, 5}
```

OUR LOCATIONS

```
set2 = {2, 3}
set3 = {1, 2, 3, 4, 5, 6}
```

Check if set2 is subset of set1

```
print(set2.issubset(set1)) # True
print(set2 <= set1) # True
```

Check if set1 is superset of set2

```
print(set1.issuperset(set2)) # True
print(set1 >= set2) # True
```

Check disjoint (no common elements)

```
set4 = {7, 8, 9}
print(set1.isdisjoint(set4)) # True
```

6. Set Functions

Function	Description
len(set)	Number of items
min(set)	Minimum value
max(set)	Maximum value
sum(set)	Sum of all items (numeric)
sorted(set)	Return sorted list
add(item)	Add single item
update(items)	Add multiple items
copy()	Create shallow copy

Table 3: Common Set Functions

Code Examples

Example 1: Basic Functions

```
numbers = {10, 50, 30, 20, 40}
```

```
print(len(numbers)) # 5
print(min(numbers)) # 10
print(max(numbers)) # 50
print(sum(numbers)) # 150
```

Example 2: sorted() Function

```
numbers = {50, 10, 30, 20, 40}
```

OUR LOCATIONS

Returns sorted list, not set

```
sorted_list = sorted(numbers)
print(sorted_list) # [10, 20, 30, 40, 50]
```

Descending order

```
desc_list = sorted(numbers, reverse=True)
print(desc_list) # [50, 40, 30, 20, 10]
```

Example 3: add() Method

```
fruits = {"Apple", "Banana"}
```

```
fruits.add("Mango")
print(fruits) # {'Apple', 'Banana', 'Mango'}
```

Adding duplicate (ignored)

```
fruits.add("Apple")
print(fruits) # {'Apple', 'Banana', 'Mango'}
```

Example 4: update() Method

```
set1 = {1, 2, 3}
```

Update with list

```
set1.update([4, 5, 6])
print(set1) # {1, 2, 3, 4, 5, 6}
```

Update with another set

```
set1.update({7, 8})
print(set1)
```

Example 5: copy() Method

```
set1 = {1, 2, 3, 4, 5}
```

Create independent copy

```
set2 = set1.copy()

set2.add(100)
print(set1) # {1, 2, 3, 4, 5} - unchanged
print(set2) # {1, 2, 3, 4, 5, 100}
```

Example 6: all() and any()

```
numbers = {2, 4, 6, 8, 10}
```

OUR LOCATIONS

Check if all items meet condition

```
print(all(x % 2 == 0 for x in numbers)) # True (all even)
```

Check if any item meets condition

```
print(any(x > 5 for x in numbers)) # True (some > 5)
```

7. Frozenset

What is Frozenset?

A frozenset is an **immutable version of set**. Once created, items cannot be added or removed.

Use Cases:

- When you need an immutable set
- To use as dictionary keys (sets can't be keys)
- To use as elements in another set

Code Examples

Example 1: Create Frozenset

From list

```
frozen = frozenset([1, 2, 3, 4, 5])  
print(frozen) # frozenset({1, 2, 3, 4, 5})
```

From set

```
my_set = {1, 2, 3}  
frozen = frozenset(my_set)  
print(frozen)
```

From string

```
frozen = frozenset("hello")  
print(frozen) # frozenset({'h', 'e', 'l', 'o'})
```

Example 2: Frozenset is Immutable

```
frozen = frozenset([1, 2, 3, 4])
```

OUR LOCATIONS

Cannot add items

`frozen.add(5) # AttributeError`

Cannot remove items

`frozen.remove(2) # AttributeError`

Cannot clear

`frozen.clear() # AttributeError`

Example 3: Frozenset Operations

```
frozen1 = frozenset([1, 2, 3, 4])
```

```
frozen2 = frozenset([3, 4, 5, 6])
```

Union

```
print(frozen1 | frozen2) # frozenset({1, 2, 3, 4, 5, 6})
```

Intersection

```
print(frozen1 & frozen2) # frozenset({3, 4})
```

Difference

```
print(frozen1 - frozen2) # frozenset({1, 2})
```

Symmetric difference

```
print(frozen1 ^ frozen2) # frozenset({1, 2, 5, 6})
```

Example 4: Frozenset as Dictionary Key

Regular set cannot be dictionary key

OUR LOCATIONS

```
my_dict = {{1, 2}: "value"} # ERROR
```

Frozenset can be dictionary key

```
frozen_key = frozenset([1, 2, 3])
my_dict = {frozen_key: "value"}
print(my_dict) # {frozenset({1, 2, 3}): 'value'}
```

Example 5: Frozenset in Set

Cannot add set to set

```
my_set = {{1, 2}, {3, 4}} # ERROR
```

Can add frozenset to set

```
frozen1 = frozenset([1, 2])
frozen2 = frozenset([3, 4])
my_set = {frozen1, frozen2}
print(my_set) # {frozenset({1, 2}), frozenset({3, 4})}
```

Example 6: Frozenset Functions

```
frozen = frozenset([10, 50, 30, 20, 40])
```

```
print(len(frozen)) # 5
print(min(frozen)) # 10
print(max(frozen)) # 50
print(sum(frozen)) # 150
print(sorted(frozen)) # [10, 20, 30, 40, 50]
```

8. Set Comprehension

What is Set Comprehension?

Concise way to create sets using a single line of code.

Syntax: {expression for item in iterable if condition}

Code Examples

Example 1: Basic Set Comprehension

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Traditional way

```
squares = set()
for x in range(1, 6):
    squares.add(x**2)
print(squares) # {1, 4, 9, 16, 25}
```

Set comprehension

```
squares = {x**2 for x in range(1, 6)}
print(squares) # {1, 4, 9, 16, 25}
```

Example 2: With Condition

Even numbers only

```
evens = {x for x in range(1, 11) if x % 2 == 0}
print(evens) # {2, 4, 6, 8, 10}
```

Numbers divisible by 3

```
div_by_3 = {x for x in range(1, 21) if x % 3 == 0}
print(div_by_3) # {3, 6, 9, 12, 15, 18}
```

Odd squares

```
odd_squares = {x**2 for x in range(1, 11) if x % 2 != 0}
print(odd_squares) # {1, 9, 25, 49, 81}
```

Example 3: String Manipulation

Unique vowels from string

```
text = "hello world"
vowels = {char for char in text if char in "aeiou"}
print(vowels) # {'e', 'o'}
```

Uppercase characters

```
text = "Python Programming"
upper_chars = {char for char in text if char.isupper()}
print(upper_chars) # {'P'}
```

Example 4: From List (Remove Duplicates)

```
numbers = [1, 2, 2, 3, 3, 3, 4, 5]
```

OUR LOCATIONS

Create set of unique numbers

```
unique = {x for x in numbers}
print(unique) # {1, 2, 3, 4, 5}
```

Unique numbers greater than 2

```
filtered = {x for x in numbers if x > 2}
print(filtered) # {3, 4, 5}
```

Example 5: Mathematical Operations

Set of remainders

```
remainders = {x % 5 for x in range(20)}
print(remainders) # {0, 1, 2, 3, 4}
```

Set of absolute differences

```
set1 = {1, 2, 3, 4, 5}
set2 = {3, 4, 5, 6, 7}
differences = {abs(a - b) for a in set1 for b in set2}
print(differences)
```

Example 6: Conditional Expression

Categorize numbers

```
numbers = range(1, 11)
categories = {"Even" if x % 2 == 0 else "Odd" for x in numbers}
print(categories) # {'Even', 'Odd'}
```

First letters

```
words = ["apple", "banana", "apricot", "berry"]
first_letters = {word[0] for word in words}
print(first_letters) # {'a', 'b'}
```

Practice Exercises

OUR LOCATIONS

Exercise 1: Set Creation

Create a set from a list with duplicates and display unique items.

Exercise 2: Set Operations

Create two sets of numbers and perform union, intersection, difference, and symmetric difference.

Exercise 3: Remove Duplicates

Given a list [1, 2, 2, 3, 3, 3, 4, 4, 4, 4], use set to remove duplicates.

Exercise 4: Set Membership

Create a set of fruits and check if specific fruits exist in the set.

Exercise 5: Set Comprehension

Create a set of squares of even numbers from 1 to 20 using set comprehension.

Exercise 6: Frozenset

Create two frozensets and use them as dictionary keys with values.

Exercise 7: Subset Check

Check if one set is a subset of another and find common elements.

Exercise 8: Unique Characters

Find all unique characters in a string using set comprehension.

Quick Reference

Operation	Syntax
Create	{1, 2, 3} or set()
Add	set.add(item)
Remove	set.remove(item) or set.discard(item)
Union	set1 set2
Intersection	set1 & set2
Difference	set1 - set2
Symmetric Diff	set1 ^ set2
Frozenset	frozenset([1, 2, 3])
Comprehension	{x for x in range(10)}

Table 4: Python Sets Quick Reference

OUR LOCATIONS

Key Points

When to use Sets:

- Remove duplicates from collection
- Test membership efficiently
- Perform mathematical set operations
- Store unique items only

Set vs Frozenset:

- Set is mutable, frozenset is immutable
- Frozenset can be dictionary key, set cannot
- Frozenset can be element in set, set cannot
- Both support same operations (union, intersection, etc.)

End of Notes

*Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: February 2026*

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune